PRODUCT REQUIREMENTS DOCUMENT

Institutional-Grade Multi-Model Deep Hedging & Execution Platform

Classification: INTERNAL — RESTRICTED

Version: 2.0.0 — Python / FastAPI Orchestration Plane

June 2026 | Supersedes v1.0.0 (Node.js / Express)

Authored by: Principal Systems Architect / Quant Infrastructure Lead

1. Executive Summary

1.1 Business Vision

The Institutional-Grade Multi-Model Deep Hedging & Execution Platform (hereafter 'the Platform') represents a convergence of cutting-edge quantitative finance, low-latency systems engineering, and modern machine learning infrastructure. Its foundational premise is that the next generation of derivatives risk management cannot be served by traditional analytical models alone — it requires a unified research-to-production pipeline that bridges PyTorch training labs, C++ execution cores, distributed orchestration planes, and institutional-grade telemetry surfaces.

The Platform is designed to operate at the intersection of two strategic imperatives: the need for ultra-low-latency risk computation demanded by market microstructure realities, and the academic and commercial imperative to deploy deep learning hedging models at production quality. Where legacy systems treat research and execution as separate silos, this architecture treats them as a single, continuously improving feedback loop.

1.2 Technical Thesis

The core technical thesis is that hybrid C++/ML architecture, governed by a gRPC-based orchestration plane and fed through lock-free ring buffers, represents the only credible path toward sub-millisecond neural inference in an institutional derivatives context. The Platform validates this thesis by benchmarking four distinct hedging regimes simultaneously:

- Native C++ Black-Scholes analytical baseline — the mathematical ground truth
- Feedforward neural network hedger — a stateless deep learning approximation
- Sequential recurrent/transformer hedger (LSTM/Transformer) — temporal context-aware hedging
- Adversarial deep hedging models — tail-risk-robust neural policies trained against worst-case market generators

By running all four regimes in parallel against identical market event streams, the Platform generates statistically rigorous comparisons of hedge quality, CVaR exposure, delta neutrality maintenance, and computational cost per decision — analytics that no existing open-source or commercial platform currently provides at this architectural depth.

1.3 Institutional Use Cases

Use Case	Description
Derivatives Risk Management	Continuous delta/gamma/vega hedging of options portfolios with real-time CVaR monitoring and automated rehedging triggers
Quant Research Infrastructure	Offline PyTorch training on historical options datasets with TorchScript export and zero-friction C++ deployment
Backtesting & Simulation	Deterministic historical replay over Parquet-stored tick data with microsecond-precision event ordering
Model Benchmarking	Simultaneous multi-model inference comparison across identical market scenarios with statistical performance attribution
Adversarial Stress Testing	Market generator models trained to expose worst-case scenarios, validating hedging strategy robustness beyond historical VaR
Latency Profiling	Per-tick latency measurement across the full ingestion-to-decision pipeline with p99/p999 latency budgets enforced

1.4 Quantitative Finance Rationale

Modern options markets demand hedging systems capable of reacting to market microstructure at sub-millisecond granularity. Traditional Black-Scholes delta hedging, while analytically tractable, makes assumptions — log-normal price dynamics, constant volatility, continuous rebalancing — that systematically fail in real markets characterized by volatility clustering, jump diffusion, and microstructure noise. Deep hedging frameworks, as originally formalized by Buehler et al. (2019), replace these analytical assumptions with learned policies trained under realistic transaction cost and market impact models, optimizing directly for tail-risk metrics such as CVaR rather than mean-squared hedging error.

The Platform operationalizes deep hedging theory at production quality: training occurs in Python/PyTorch with full experiment tracking, compiled models are deployed as TorchScript artifacts inside the C++ core, and inference runs within the same lock-free execution loop that processes market ticks. This eliminates the research-production gap that plagues institutional ML deployments.

1.5 Why Hybrid C++ + ML Architecture Matters

Python's GIL and garbage collector make it structurally incapable of meeting sub-millisecond latency requirements in a high-frequency execution context. Conversely, pure C++ systems lack the ergonomic ML tooling required for rapid model iteration. The hybrid architecture resolves this tension architecturally: Python owns the training and research lifecycle, C++ owns the inference and execution lifecycle, and TorchScript provides the cross-language boundary. LibTorch — the C++ distribution of PyTorch — enables loading and executing serialized model graphs inside the C++ process without Python runtime overhead, achieving near-identical inference performance to CUDA-accelerated native PyTorch.

1.6 Why CVaR Optimization Matters

Value-at-Risk (VaR) measures a loss threshold at a given confidence level but provides no information about the expected magnitude of losses beyond that threshold. Conditional Value-at-Risk (CVaR), also called Expected Shortfall, directly measures the expected loss in the tail — precisely the regime that matters most for solvency and margin requirements. Training hedging models with CVaR loss functions rather than MSE produces policies that are demonstrably more robust to extreme market events, at the cost of slightly higher expected costs in benign conditions. For institutional risk management, this tradeoff is unambiguously correct.

1.7 Why Adversarial Training Matters

Historical backtesting is limited by the distribution of observed market conditions. Adversarial training — wherein a market generator model is trained to maximize hedging loss while the hedging model trains to minimize it — creates a minimax game that generates worst-case scenarios beyond the historical distribution. This produces hedging policies that are robust not just to observed volatility regimes but to adversarially constructed stress scenarios, providing a stronger guarantee of solvency under novel market conditions.

1.8 Why Infrastructure-First Strategy is Strategically Correct

Many quantitative research projects fail at productionization, not because their models are wrong, but because their infrastructure cannot support the operational requirements of live deployment. By designing the execution core, IPC layer, orchestration plane, and observability stack as first-class concerns before optimizing model architectures, the Platform ensures that every model improvement is immediately deployable without infrastructure rework. Infrastructure investment pays compound returns across the entire research timeline.

2. Product Vision

2.1 Long-Term Platform Direction

The Platform is architected from inception as an institutional-grade system, not a prototype to be rewritten. Every architectural decision — lock-free buffers, deterministic replay, TorchScript portability, gRPC typed contracts — is made with production deployment as the frame of reference. The long-term trajectory moves through three phases: (1) a validated research-to-inference pipeline for derivatives hedging; (2) a live simulation environment with real-time market data feeds; (3) a smart order routing and execution management system capable of submitting orders to live venues.

2.2 Live Trading Ambitions

The Platform's architecture is explicitly designed to accommodate live trading by incrementally replacing simulated components with real exchange connectivity. The ZeroMQ market feed consumer can be re-pointed from synthetic tick generators to real FIX/ITCH/PITCH feed adapters. The execution acknowledgment path already carries the latency characteristics required for algorithmic execution. The gRPC streaming layer, ARQ orchestration, and FastAPI WebSocket broadcast layer require no modification whatsoever for the live trading use case.

2.3 Research-to-Production Pipeline Philosophy

The Platform enforces a strict separation of concerns between the research environment and the production runtime. The Python ML lab (research/brain_layer_ml) has no runtime dependency on the C++ core — it reads static Parquet data, trains models, logs telemetry to Weights & Biases, and writes TorchScript artifacts to disk. The C++ core has no knowledge of Python — it reads .pt files from a model registry, loads them via LibTorch, and executes inference inside its zero-allocation execution loop. The TorchScript artifact is the only coupling between these two worlds, and it is immutable once deployed — ensuring that the model running in production is byte-for-byte identical to the model validated in research.

2.4 Comparison Against Existing Quant Architectures

Dimension	Typical Quant Platform	This Platform
Execution Core	Python or Java-based, GC pauses	C++20, zero-allocation hot path
IPC	REST, message queues	gRPC Protobuf, lock-free SPSC
ML Integration	Separate offline batch jobs	TorchScript embedded in C++ core
Risk Metrics	EOD batch computation	Real-time per-tick CVaR streaming
Observability	Log file aggregation	Structured JSON + correlation IDs
Replay	Best-effort approximate	Deterministic microsecond-exact
Adversarial Testing	Absent or manual	Integrated minimax training loop

2.5 Why Deterministic Infrastructure is Critical

Non-deterministic systems make it impossible to distinguish model behavior from infrastructure noise. If a backtest produces different P&L on identical inputs, it is impossible to attribute the difference to model changes versus scheduling jitter, network latency variation, or memory allocation non-determinism. Deterministic replay — achieved through timestamped event logs, seeded random number generators, and

synchronized thread execution — is not a nice-to-have; it is a prerequisite for scientifically valid model comparison.

3. System Objectives

3.1 Primary Objectives

- Deploy a unified C++/Python platform for multi-model options hedging simulation and backtesting
- Benchmark Black-Scholes, feedforward NN, LSTM/Transformer, and adversarial deep hedging models simultaneously
- Achieve sub-millisecond tick-to-decision latency in the C++ execution core under load
- Provide real-time CVaR and Greeks streaming to the frontend terminal via WebSocket
- Enable deterministic historical replay for reproducible backtesting

3.2 Secondary Objectives

- Support multiple dataset sources: Databento, IBKR, ThetaData, Polygon, Kaggle, Optiver, BANKNIFTY
- Provide structured JSON logging (loguru) with correlation ID propagation across all service boundaries
- Implement ARQ-based async job management for long-running backtests
- Persist strategy metadata, backtest results, and model versions in PostgreSQL via SQLModel / Alembic
- Expose a FastAPI REST control surface for backtest configuration, job management, and model registry operations

3.3 Research Objectives

- Implement differentiable CVaR loss function in PyTorch with gradient verification
- Train and validate LSTM and Transformer architectures on historical options tick data
- Implement adversarial market generator as a minimax training partner to the hedging network
- Export trained models via TorchScript JIT tracing for C++ inference compatibility
- Track all experiments via Weights & Biases with hyperparameter sweeps

3.4 Infrastructure Objectives

- Implement lock-free SPSC and MPSC ring buffers in C++20 with cache-line alignment
- Pin execution threads to dedicated CPU cores to eliminate scheduler jitter
- Implement zero-copy gRPC streaming between C++ engine and Python orchestrator using grpc.aio
- Containerize all services via Docker Compose with reproducible uv / vcpkg / pnpm lock-file builds
- Maintain strict Python virtual environment separation between apps/backend and research/brain_layer_ml

3.5 Technical Stack Summary

Layer	Technology — Primary Purpose
Low-Latency Core	C++20 (GCC / Clang) — Execution loops, order routing simulation, real-time math engines
Linear Algebra & Math	LibTorch C++ API, lets_be_rational — Neural inference and Black-Scholes/Greek utilities
Inter-Process Comm (IPC)	gRPC & Protocol Buffers — Typed bidirectional streaming between C++

	and Python
Transport Layer	ZeroMQ (ZMQ) — Lock-free PUB/SUB for raw market tick ingestion
Memory Architecture	Atomic SPSC / MPSC Ring Buffers — Zero-allocation lock-free tick processing
Backend & Orchestration	Python 3.11, FastAPI, uvicorn — REST API, WebSocket broadcast, gRPC client
Async gRPC Client	grpcio / grpc.aio — Asyncio-native async iterator streaming of EngineStateUpdates
Task Queue & Async Workers	ARQ & Redis — Async Python queue for backtest jobs, retry, and progress
Database ORM	SQLModel (SQLAlchemy 2.0 + Pydantic v2) + asyncpg + Alembic — Async PostgreSQL
Historical Data Store	Apache Parquet, AWS S3 — Columnar compressed options tick histories
Frontend UI Terminal	Next.js, React, Tailwind CSS, Shadcn UI — Dashboard and real-time state management
Real-Time Visualization	Recharts, Zustand — Streaming P&L distributions, CVaR curves via WebSocket
Structured Logging	loguru — JSON-serialized logs with correlationId ContextVar binding
Dependency Management	uv (Python), pnpm (TypeScript/frontend only), vcpkg (C++)
Testing	GoogleTest (C++), Pytest + pytest-asyncio + httpx (Python), Vitest (TS)

3.6 Performance Objectives

Metric	Target
Tick-to-decision latency (p50)	< 50 microseconds
Tick-to-decision latency (p99)	< 200 microseconds
gRPC stream throughput	> 50,000 updates/second
WebSocket broadcast latency	< 5 milliseconds
Frontend render rate	60 Hz (RAF-driven)
C++ ring buffer throughput	> 10 million ops/second
PostgreSQL async write throughput	> 5,000 rows/second batched
Backtest replay accuracy	Microsecond timestamp fidelity

4. Functional Requirements

FR-01: Historical Backtesting

Purpose: Enable deterministic replay of historical market events against all hedging models simultaneously, producing comparable P&L distributions and risk metrics.

Workflow: User submits POST /api/run_backtest with strategy configuration (models, date range, instrument, notional). FastAPI route validates the Pydantic request body, generates a UUID correlationId, and enqueues an ARQ job via await redis.enqueue_job('run_backtest_job', job_payload, _job_id=correlationId). The ARQ worker deserializes configuration, fetches metadata from PostgreSQL via AsyncSession, resolves Parquet file paths, and initiates a grpc.aio server-streaming call to the C++ engine. The engine replays ticks in chronological order, computes Black-Scholes and LibTorch model outputs for each tick, and streams P&L/risk state back. The ARQ handler iterates the async response iterator and broadcasts each EngineStateUpdate via FastAPI WebSocket to subscribed frontend clients.

Edge Cases: Parquet file corruption or schema mismatch triggers graceful job failure with structured error payload. Tick timestamps with microsecond gaps < 1µs are coalesced. Overnight gaps in tick data are handled via configurable session boundary detection. gRPC stream disconnection during replay triggers ARQ job retry with checkpoint resume.

Data Dependencies: Parquet tick files (S3/local), strategy configuration record (PostgreSQL), model registry paths (.pt artifacts on disk)

Expected Latency Profile: Job submission: < 10ms. First tick broadcast: < 500ms after job dequeue. Sustained replay throughput: up to 1M ticks/second in fast-forward mode.

Validation Logic: P&L at expiry must equal intrinsic value minus accumulated hedging costs within floating-point tolerance. Greeks at option expiry must converge to discontinuous payoff delta.

FR-02: Live Streaming Simulation

Purpose: Simulate real-time market conditions using synthetic tick generation or live feed adapters, enabling continuous model evaluation without historical dataset constraints.

Workflow: Live simulator in C++ generates GBM/Heston ticks at configurable frequency and injects them into the SPSC tick buffer via the same path as historical ticks. The execution loop is feed-agnostic — it processes whatever arrives in the buffer without distinguishing between historical and live events. Strategy state persists across the simulation session and is exposed identically to the backtest path.

Edge Cases: Synthetic tick generation must be seeded deterministically for reproducibility. Feed rate throttling must prevent buffer overflow under model computation backpressure. Session restart must clear all transient hedge positions and reset P&L accounting.

FR-03: gRPC Streaming IPC

Purpose: Provide a strictly typed, bidirectional streaming control plane between the C++ execution core and the Node.js orchestration layer.

Workflow: The C++ gRPC server exposes a StartBacktest streaming RPC that accepts a StreamRequest and returns a stream of EngineStateUpdate messages. The Python grpc.aio client opens the stream via stub.StartBacktest(request, metadata=[...]), iterates the async response iterator in the ARQ worker coroutine, and pipes each update to the FastAPI WebSocket broadcast layer. The reverse direction carries control commands (pause, resume, stop, model swap) via a CommandRequest stream emitted from the FastAPI backend.

Edge Cases: gRPC stream backpressure must be handled via flow control — the C++ outbound SPSC buffer prevents unbounded accumulation. Protobuf schema evolution must be backward-compatible across engine and orchestrator versions.

Expected Latency Profile: gRPC framing overhead < 10µs per message. End-to-end IPC round-trip < 500µs.

FR-04: CVaR Computation

Purpose: Compute per-tick Conditional Value-at-Risk at configurable confidence levels (95%, 99%, 99.9%) across the running P&L distribution for each hedging model.

Workflow: The execution loop maintains a sliding window of P&L observations (configurable window size, default 1000 ticks). After each tick, CVaR is computed as the average of the worst-case tail observations below the VaR threshold. CVaR deltas are included in the EngineStateUpdate payload alongside Greeks and P&L.

Edge Cases: Window initialization period (insufficient observations) returns NaN with explicit flag. CVaR computation must not block the execution loop — it is computed inline with O(1) amortized complexity using a sorted insertion structure or approximate histogram.

FR-05: Greeks Calculation

Purpose: Compute Delta, Gamma, Vega, Theta, and Rho for each option position at every tick, providing the mathematical foundation for hedging decision-making.

Workflow: Black-Scholes analytical Greeks are computed via closed-form expressions using the black_scholes.hpp utility. For ML models, effective delta is computed via finite-difference differentiation of the model output with respect to spot price perturbation. Greeks are included in the EngineStateUpdate and used downstream for delta-neutrality maintenance.

Validation Logic: Put-call parity must hold within floating-point precision. Delta of deep ITM call must converge to 1.0; deep OTM call must converge to 0. Gamma must be non-negative for long positions.

FR-06: WebSocket Broadcasting

Purpose: Deliver real-time P&L, CVaR, Greeks, and model comparison data to all subscribed frontend clients at up to 60 Hz.

Workflow: The FastAPI WebSocket server maintains a job_subscribers registry: a dict[str, set[WebSocket]] mapping jobId to connected clients. Incoming gRPC EngineStateUpdate Protobuf messages are deserialized via google.protobuf.json_format.MessageToDict and broadcast to all subscribers of the relevant jobId via asyncio.gather(*[ws.send_json(payload) for ws in subscribers]). Disconnected clients raise WebSocketDisconnect and are removed from the registry. Broadcast is non-blocking within the asyncio event loop — a single slow client cannot block delivery to other subscribers.

Throughput Target: > 10,000 messages/second per WebSocket server instance.

FR-07: Model Registry & TorchScript Loading

Purpose: Manage the lifecycle of compiled ML model artifacts, enabling hot-swap of model versions without C++ process restart.

Workflow: The LibTorch registry in the C++ core maintains a map of model identifiers to loaded torch::jit::Module instances. On receipt of a LoadModel command via gRPC, the registry loads the .pt artifact from the configured path, validates the input/output tensor shapes against the registry schema, and registers the module as the active inference target. The execution loop queries the registry by model ID before each inference call.

FR-08: Experiment Telemetry & Weights & Biases Integration

Purpose: Provide complete experiment lineage, hyperparameter tracking, and training curve visualization for all ML training runs.

Workflow: The PyTorch training loop calls wandb.log() at each training step with loss, validation metrics, gradient norms, and CVaR estimates. Hyperparameters are logged at run initialization. Model checkpoints are saved locally and optionally synced to W&B Artifacts. The exported TorchScript model is tagged with the W&B run ID for full traceability.

FR-09: Adversarial Market Generation

Purpose: Train a market generator adversary that produces worst-case tick sequences to expose hedging model weaknesses.

Workflow: The adversarial generator is a separate neural network that takes market state as input and outputs tick perturbations. It is trained in a minimax loop: the generator maximizes hedging loss, the hedger minimizes it. The generator is constrained to produce statistically plausible tick sequences (bounded jump sizes, realistic autocorrelation) via regularization terms in its loss function.

FR-10: ARQ Job Orchestration

Purpose: Manage long-running backtest and training jobs with async execution, retry logic, and progress reporting.

Workflow: FastAPI route handler enqueues jobs via arq redis.enqueue_job() with correlationId as job ID. ARQ worker coroutines consume jobs from Redis and execute the backtest pipeline asynchronously. Each job reports progress via ctx['job'].update_progress(pct). Job status (queued/in_progress/complete/failed) is queryable via arq.jobs.Job.status(). Completed jobs persist result summaries to PostgreSQL via AsyncSession. WebSocket subscribers receive progress events relayed from the ARQ progress listener background task.

5. Non-Functional Requirements

NFR-01: Latency Guarantees

Latency Metric	Target Budget
C++ tick ingestion to ring buffer push	< 1 microsecond
Ring buffer pop to Black-Scholes evaluation	< 5 microseconds
LibTorch inference (feedforward)	< 50 microseconds
LibTorch inference (LSTM/Transformer)	< 200 microseconds
gRPC state stream frame emission	< 10 microseconds overhead
Node.js gRPC receive to WS broadcast	< 1 millisecond
WebSocket message to Zustand state update	< 2 milliseconds
React re-render triggered by Zustand	< 16 milliseconds (60 Hz)

NFR-02: Zero-Copy Architecture

The C++ execution hot path is designed as a zero-allocation loop. All memory required for tick processing, model inference, and state emission is pre-allocated at engine startup. Tick structures are written directly into ring buffer slots — no heap allocation occurs on the critical path. LibTorch tensor buffers are pre-allocated and reused across inference calls. gRPC streaming uses pre-allocated Protobuf message objects with field reuse.

NFR-03: Deterministic Replay

Historical replay must produce byte-identical output given identical input. This requires: (1) a seeded pseudo-random number generator for any stochastic simulation components; (2) single-threaded execution of the replay loop (no concurrent tick processing); (3) IEEE 754 floating-point operations in the same order on the same hardware; (4) no dependency on system time during replay — all temporal logic must reference the tick timestamp. Determinism is validated via a test suite that runs identical replays twice and compares SHA-256 checksums of all emitted state updates.

NFR-04: Fault Tolerance

The C++ engine is designed to survive individual tick processing failures without crashing. Malformed ticks are logged with full diagnostic context and dropped. gRPC stream reconnection is handled transparently by the Python grpc.aio client with exponential backoff. ARQ job failure triggers configurable retry with preserved checkpointing. PostgreSQL write failures trigger buffered retry with circuit breaker pattern to prevent cascading queue saturation.

NFR-05: Observability & Structured Logging

Every service boundary emits structured JSON logs via loguru (Python backend) and spdlog-compatible structured output (C++). All log entries carry: (1) ISO 8601 timestamp with microsecond precision; (2) service identifier; (3) correlation ID (via ContextVar in Python); (4) severity level; (5) structured payload fields. No unstructured string concatenation is used in production log paths. Log levels are configurable per service via LOG_LEVEL environment variable without restart.

NFR-06: CPU Affinity & Thread Isolation

The C++ engine assigns threads to CPU cores at startup using pthread_setaffinity_np (Linux) or SetThreadAffinityMask (Windows). Core assignments are configurable via environment variables. Execution loop thread is assigned to an isolated core with IRQ affinity configured to avoid interruptions. ZeroMQ consumer and gRPC I/O threads are assigned to separate cores. NUMA topology is respected — all threads in the engine process share the same NUMA node to avoid cross-node memory access.

NFR-07: Bounded Memory Behavior

Ring buffer capacities are fixed at compile time. LibTorch tensor buffers are pre-allocated. ARQ queue depth is bounded by max_jobs per WorkerSettings and configurable job_timeout eviction. FastAPI WebSocket client buffers are bounded — WebSocketDisconnect exceptions evict slow clients rather than accumulating unbounded backlog. PostgreSQL connection pool size is fixed via pool_size/max_overflow parameters. Redis memory limit is configured with LRU eviction policy.

NFR-08: Horizontal Scalability

ARQ workers are stateless coroutines and horizontally scalable by running additional arq worker processes against the same Redis-backed queue. Multiple uvicorn worker processes can handle WebSocket connections with Redis PubSub as the fan-out backbone. C++ engine instances are stateful and not horizontally scalable in V1 — vertical scaling via CPU affinity and SIMD optimization is the primary performance lever.

NFR-09: Reproducibility Guarantees

Docker images are built from pinned base image digests. vcpkg dependency versions are pinned in vcpkg.json. Python dependencies are pinned in requirements.txt with hash verification. Node.js dependencies are pinned in pnpm-lock.yaml. All environment variables affecting behavior are documented and validated at startup. CI/CD pipeline reproduces the build environment deterministically.

6. System Architecture

6.1 Macro Architecture

The Platform is organized into four primary execution domains, each with a distinct runtime context, responsibility boundary, and deployment strategy. These domains communicate through strictly typed interfaces — no implicit shared memory, no untyped HTTP JSON blobs on the critical path. A deliberate design decision in this architecture is to maximize code reuse from the existing OTrader v2 codebase, whose Python/FastAPI backend provides a production-validated gRPC client, WebSocket broadcast layer, and PostgreSQL persistence layer. The orchestration domain is therefore implemented in Python rather than Node.js, collapsing the Number of distinct runtime environments from three (C++, Node.js, Python) down to two (C++, Python) and eliminating an entire language boundary from the operational surface.

Four Execution Domains

Domain 1 — C++ Execution Core: Low-latency market simulation, mathematical pricing, and LibTorch inference

Domain 2 — Python/FastAPI Orchestration Plane: Job management, async gRPC client, WebSocket broadcast, REST API

Domain 3 — Next.js Frontend Terminal: Real-time state visualization, configuration, and telemetry

Domain 4 — Python ML Research Lab: Offline training, model compilation, and experiment tracking

The Python orchestration plane and Python ML research lab share a language runtime but remain strictly separated as independent deployable units — the orchestration plane is a long-running FastAPI/uvicorn process with live network dependencies, while the research lab is a batch-execution process with no network dependencies. They share no module imports across process boundaries; the only coupling is the .pt model artifact written to a shared filesystem path.

6.2 C++ Execution Core

6.2.1 Responsibility

The execution core is the performance-critical heart of the platform. It is responsible for: tick ingestion from ZeroMQ subscriptions; lock-free buffering of market events; deterministic execution of Black-Scholes pricing and Greeks computation; LibTorch model inference for neural hedging models; CVaR streaming state computation; and gRPC state emission to the Python orchestrator. It has no network dependencies beyond ZeroMQ and gRPC, no database access, and no filesystem I/O on the critical path.

6.2.2 Internal Architecture

The core operates four logical thread roles: (1) ZMQ Subscriber Thread — blocks on zmq_recv, decodes tick payloads, pushes to SPSC tick buffer using atomic operations; (2) gRPC Command Thread — blocks on gRPC completion queue, decodes CommandRequest messages, pushes to MPSC command buffer; (3) Main Execution Loop Thread — the only thread that reads from both input buffers, executes all pricing/inference logic, and writes to the outbound SPSC state buffer; (4) gRPC Stream Thread — blocks on outbound SPSC buffer, pops EngineStateUpdate structs, serializes to Protobuf, and emits via gRPC server-side streaming RPC.

6.2.3 Threading Considerations

The main execution loop thread must never block. All buffer operations are non-blocking atomic CAS operations. LibTorch inference on CPU is synchronous but bounded — inference time is profiled and monitored per model, with configurable timeout eviction. The execution loop implements a configurable spin-

wait strategy: tight spin for sub-millisecond expected latency regimes, yield-on-empty for low-frequency simulation modes.

6.2.4 Performance Constraints

All allocations on the hot path are forbidden after engine initialization. std::vector, std::string, std::map, and any STL container with heap allocation are prohibited in the execution loop body. Arithmetic operations use fixed-size arrays and stack-allocated structs. SIMD intrinsics (SSE4.2/AVX2) are used for batch Black-Scholes computation where applicable.

6.2.5 Failure Modes

Buffer overflow (producer faster than consumer): handled by dropping ticks with monotonically increasing drop counter logged. gRPC connection loss: outbound buffer continues filling; emission thread retries connection. LibTorch inference failure (NaN/Inf output): model output is rejected, Black-Scholes fallback activates, event logged.

6.3 gRPC IPC Layer

The gRPC layer serves as the sole typed communication interface between the C++ engine and the Python FastAPI orchestrator. It provides two streaming RPCs: StartBacktest (server-side streaming from C++ to Python) and SendCommand (client-side streaming from Python to C++). All message types are defined in engine_service.proto and compiled to language-specific bindings via protoc — generating both C++ bindings (grpc_cpp_plugin) and Python stubs (grpc_tools.protoc). The proto file is the single source of truth for the IPC contract, versioned in the repository alongside both engine and backend sources. The Python gRPC client uses the grpcio library (grpc.aio channel for asyncio-native operation), meaning the async FastAPI event loop manages gRPC I/O without blocking threads.

6.4 Python / FastAPI Orchestration Plane

The Python FastAPI backend (apps/backend) operates as the macro-level control surface. It is architected in direct continuity with the OTrader v2 backend design: a layered structure separating HTTP API routes (src/api/), domain service logic (src/services/), infrastructure integration (src/infra/), and Protobuf stubs (src/proto/). FastAPI is chosen as the HTTP framework because: (1) it is natively async — route handlers are coroutines that share the uvicorn event loop with gRPC stream consumers and WebSocket broadcast tasks without thread-pool context switching; (2) Pydantic model validation at route boundaries provides the same request schema enforcement that Express + Zod provides in the Node.js alternative; (3) OpenAPI documentation is generated automatically from Pydantic schemas, eliminating separate API documentation overhead.

The application lifecycle is managed by FastAPI's lifespan context manager (asynccontextmanager), mirroring the OTrader v2 _otrader_lifespan pattern. At startup, the lifespan: (1) initialises the grpc.aio channel to the C++ engine; (2) launches the log_queue_processor background task for WebSocket log forwarding; (3) starts the live_status heartbeat coroutine. At shutdown, all tasks are cancelled cleanly and the gRPC channel is closed. This lifecycle management is deterministic and testable — unlike Node.js process-level signal handlers, FastAPI's lifespan is a first-class framework construct.

The internal module boundaries follow the OTrader v2 call flow pattern: HTTP request enters a route in src/api/ (protocol and input parsing only); route calls a service in src/services/ (business rules, response shaping, error semantics); service calls infrastructure in src/infra/ (gRPC client, async task queue, database, state). No direct cross-layer calls are permitted — the layering is enforced by code review and linting rules.

Layer	Responsibility & Key Modules
src/api/engine.py	Gateway connect/disconnect, market data start/stop, strategy lifecycle endpoints, holdings, logs
src/api/backtest.py	Backtest file listing, strategy class query, run_backtest, cancel endpoints

src/api/system.py	System health, DB views (orders/trades, contracts), live engine status relay
src/services/main.py	Gateway/market/orders-trades logic; wraps EngineClient; shapes responses
src/services/strategy.py	Strategy lifecycle rules; validates parameters before gRPC dispatch
src/services/backtest.py	Param validation, default injection, subprocess result shaping
src/infra/remote_client.py	grpc.aio EngineClient — wraps all C++ gRPC calls; converts proto to dicts
src/infra/backtest_runner.py	Resolves parquet paths; spawns entry_backtest subprocess; parses stdout JSON
src/infra/state.py	AppState singleton: live client, backtest proc handle, WebSocket client sets
src/infra/websockets.py	Log and strategy WebSocket handlers; log_queue_processor broadcast task
src/proto/	grpcio-generated stubs from engine_service.proto

6.5 Python Async Task Queue — ARQ / Redis

Long-running historical backtest jobs are managed by an async Python task queue backed by Redis. The selected implementation is ARQ (Async Redis Queue), chosen over Celery for three reasons: (1) ARQ is natively asyncio-compatible — worker coroutines run inside the same event loop model as FastAPI, eliminating the impedance mismatch of mixing asyncio application code with Celery's multiprocessing worker model; (2) ARQ's job lifecycle (enqueue, dequeue, progress, complete, retry) maps directly onto the AppState.backtest pattern already established in OTrader v2; (3) ARQ's Redis serialization uses msgpack by default, offering lower serialization overhead than Celery's pickle or JSON for numeric job payloads.

ARQ organizes jobs into named function queues. The Platform defines three queues: backtest_queue (historical replay jobs, concurrency=1 per engine instance), training_notify_queue (notifications to trigger ML re-evaluation after new data), and report_queue (async result aggregation and PDF export). Job data is limited to 1MB. Job progress is reported via arq.jobs.Job.progress and relayed to WebSocket subscribers via the log_queue_processor. ARQ workers are started as a separate process alongside the FastAPI application, configured via a WorkerSettings dataclass that specifies Redis connection parameters, queue names, and retry policies.

6.6 PostgreSQL Persistence Layer — SQLModel / SQLAlchemy

PostgreSQL persists: strategy configurations, backtest run results, model registry entries, tick archive metadata, and risk snapshots. The ORM layer uses SQLModel — a library that unifies SQLAlchemy table definitions with Pydantic schema declarations. SQLModel table classes serve as both the database schema definition (via SQLAlchemy Core) and the Pydantic response model at API boundaries (via Pydantic BaseModel inheritance), eliminating the dual-definition overhead of maintaining separate SQLAlchemy models and Pydantic response schemas.

Database connectivity uses SQLAlchemy's async engine (create_async_engine with asyncpg driver) to ensure all database I/O is non-blocking within the FastAPI async event loop. Sessions are managed via AsyncSession with contextvar-scoped session lifecycle — each request obtains a session from the connection pool, uses it for the duration of the request, and releases it on completion. Connection pooling parameters (pool_size=10, max_overflow=20) are configurable via environment variables. Schema migrations are managed by Alembic, with migration scripts generated from SQLModel table class changes and versioned in the repository.

6.7 WebSocket Broadcast Layer — FastAPI Native

FastAPI's native WebSocket support (fastapi.WebSocket) is used for all real-time client communication, eliminating the need for a separate WebSocket library. The infra/websockets.py module maintains two client registries: log_clients (Set[WebSocket] for the /ws/logs endpoint) and strategy_clients (Set[WebSocket] for the /ws/strategies endpoint). The log_queue_processor is an asyncio background Task that consumes from AppState.live.log_queue (an asyncio.Queue) and broadcasts each log entry to all connected log_clients via await ws.send_json().

For backtest state broadcasting — where payloads are deserialized from gRPC EngineStateUpdate Protobuf messages — a separate job_stream_processor task is created per active backtest job. This task: (1) opens the grpc.aio server-streaming call to the C++ engine; (2) iterates the async response iterator in an async for loop; (3) deserializes each EngineStateUpdate Protobuf message to a Python dict via the proto message's field accessors; (4) serializes to JSON and broadcasts to all subscribers of the relevant jobId via await ws.send_json(). The async for loop over the gRPC stream iterator is naturally non-blocking — it yields control to the event loop while awaiting the next message, allowing concurrent WebSocket broadcasts and REST request handling on the same uvicorn worker thread.

Slow client handling: if await ws.send_json() raises a WebSocketDisconnect exception, the client is removed from the registry. No per-client buffer draining timeout is required — FastAPI's WebSocket implementation handles the underlying TCP send buffer; the application layer only needs to handle disconnect events.

6.8 Next.js Frontend Terminal

The frontend is a server-side-rendered React application with App Router. It serves as a pure presentation layer — no financial computations occur in the browser. State management is handled by Zustand slices that receive WebSocket updates and expose derived views to React components. Recharts canvases are driven by Zustand subscriptions and re-render at up to 60 Hz via requestAnimationFrame scheduling. Shadcn UI components provide the configuration panel and control surfaces. The frontend communicates with the FastAPI backend over HTTPS REST (for configuration and job submission) and native browser WebSocket (for streaming updates) — no protocol changes are required at the frontend layer as a result of the Node.js-to-Python backend migration.

6.9 ML Research Workspace

The Python research workspace is deliberately isolated from the runtime system. It reads Parquet files, trains models, exports TorchScript artifacts, and has no network dependency on any other Platform component during training. Its only output that matters to the runtime is the .pt model file written to the shared artifacts directory. The research workspace and the FastAPI orchestration plane share a Python interpreter version (pinned via .python-version) but have entirely separate virtual environments — the research environment carries torch, numpy, pandas, and wandb; the orchestration environment carries fastapi, grpcio, sqlmodel, and arq. Cross-contamination of dependencies is prevented by enforcing separate venv activation paths in both Docker build stages and developer setup scripts.

7. Deep Dive — Low-Latency C++ Engine

7.1 Lock-Free Data Structures

The Platform's lock-free architecture is built on two canonical primitives: Single-Producer Single-Consumer (SPSC) ring buffers and Multi-Producer Single-Consumer (MPSC) ring buffers. Both are implemented in C++20 using std::atomic with explicit memory ordering annotations.

7.1.1 SPSC Ring Buffer

The SPSC buffer (spsc_ring_buffer.hpp) is a power-of-two-sized circular array where the producer and consumer each own a single pointer. The producer atomically writes to the slot at (write_idx % capacity) and increments write_idx with std::memory_order_release. The consumer reads from (read_idx % capacity) after checking write_idx with std::memory_order_acquire. No mutex, no condition variable, no OS syscall. Push and pop are O(1) with a single atomic read-modify-write.

```
template<typename T, size_t N>
class SPSCRingBuffer {
  alignas(64) std::atomic<size_t> write_idx{0};
  alignas(64) std::atomic<size_t> read_idx{0};
  alignas(64) T slots[N];
  bool push(const T& val) noexcept {
    auto w = write_idx.load(std::memory_order_relaxed);
    if (w - read_idx.load(std::memory_order_acquire) == N) return false;
    slots[w % N] = val;
    write_idx.store(w + 1, std::memory_order_release);
    return true; }
};
```

7.1.2 Why Lock-Free Matters

A mutex-based queue incurs at minimum two syscall round-trips per lock/unlock pair (futex_wait / futex_wake on Linux), costing 500ns–5µs depending on contention level. In a high-frequency tick processing loop targeting < 1µs per tick, this is structurally incompatible. The SPSC buffer achieves the same producer-consumer synchronization with two cache-coherent atomic operations — approximately 10–50ns on modern x86 hardware.

Beyond raw latency, mutex-based queues introduce priority inversion and unbounded latency tails under OS scheduling jitter. A thread holding a mutex can be preempted mid-critical-section, stalling all waiters for the duration of the preemption window. Lock-free operations are immune to this failure mode — progress is guaranteed as long as the hardware cache coherence protocol makes forward progress.

7.2 False Sharing Prevention

False sharing occurs when two variables sharing a cache line are written by different threads — each write invalidates the other thread's cache copy even though the variables are logically independent. In the SPSC buffer, write_idx and read_idx are each aligned to separate 64-byte cache lines (alignas(64)) to prevent the consumer's read of read_idx from invalidating the producer's write_idx cache line and vice versa. This is not a micro-optimization — false sharing can degrade throughput by 10x in cache-coherent multi-core architectures.

7.3 Zero-Allocation Execution Loops

The main execution loop is the most performance-critical code in the entire Platform. It is written to violate zero allocation invariants: no new/malloc on the hot path, no std::string construction, no STL container insertions. All working memory is stack-allocated or pre-allocated in fixed-size pools at engine startup. LibTorch inference tensors are pre-allocated with the correct shape and dtype; their data buffers are populated in-place from the tick struct fields without intermediate copies.

7.4 CPU Affinity Pinning

Thread-to-core assignment is performed at engine startup using pthread_setaffinity_np with a CPU set constructed from environment variable configuration. The recommended assignment for a 6-core system is: Core 0 reserved for OS; Core 1 for ZMQ subscriber; Core 2 for gRPC I/O; Core 3 for main execution loop (isolated, no IRQ); Cores 4–5 for gRPC emission and auxiliary tasks. Affinity pinning eliminates cross-core cache migration overhead — the execution loop's working set remains resident in L1/L2 of its dedicated core.

7.5 gRPC Async Completion Queues

The C++ gRPC server uses the async API (grpc::ServerCompletionQueue) rather than the synchronous API. This is critical for two reasons: (1) the synchronous API blocks a thread per RPC call, wasting resources in a high-concurrency streaming scenario; (2) the async API allows the gRPC I/O thread to multiplex multiple stream states on a single OS thread via completion queue polling, compatible with the Platform's thread budget constraints.

7.6 Market Tick Processing Lifecycle

Each tick traverses the following pipeline: (1) ZMQ socket recv() populates a raw byte buffer on the ZMQ thread stack; (2) tick struct is deserialized from the buffer and pushed to the SPSC tick buffer via atomic write_idx increment; (3) execution loop reads the tick from the SPSC buffer via atomic read_idx; (4) Black-Scholes functions are called with tick fields as arguments, returning Greeks and theoretical price; (5) LibTorch inference tensors are populated from tick fields and all registered models are evaluated in sequence; (6) CVaR sliding window is updated and computed; (7) EngineStateUpdate struct is populated on the stack and pushed to the outbound SPSC buffer; (8) gRPC emission thread pops the state update, serializes to Protobuf, and calls ServerWriter::Write().

8. Deep Dive — Quantitative Finance Layer

8.1 Black-Scholes Framework

The Black-Scholes-Merton model prices European options under the assumption of log-normal asset price dynamics, constant volatility, no dividends, and frictionless markets. While these assumptions are known to be violated in practice, the model remains the universal reference frame for options pricing — implied volatility, the primary observable derived from market prices, is defined as the volatility parameter that makes the BSM formula reproduce the market price.

8.1.1 Call and Put Pricing Formulas

For a European call option with spot S, strike K, risk-free rate r, time to expiry T, and volatility σ:

Black-Scholes Pricing

$d_1 = [\ln(S/K) + (r + \sigma^2/2)T] / (\sigma\sqrt{T})$

$d_2 = d_1 - \sigma\sqrt{T}$

Call Price $C = S \cdot N(d_1) - K \cdot e^{(-rT)} \cdot N(d_2)$

Put Price $P = K \cdot e^{(-rT)} \cdot N(-d_2) - S \cdot N(-d_1)$

where $N(\cdot)$ is the standard normal CDF

8.2 Greeks

Greek	Formula / Interpretation
Delta (Δ)	$\partial C/\partial S = N(d_1)$ for calls. Sensitivity of option price to spot price change. Primary hedge ratio.
Gamma (Γ)	$\partial^2 C/\partial S^2 = N'(d_1)/(S\sigma\sqrt{T})$. Rate of change of delta. Convexity cost of delta hedging.
Vega (ν)	$\partial C/\partial \sigma = S\sqrt{T} \cdot N'(d_1)$. Sensitivity to volatility. Critical in volatility regime shifts.
Theta (Θ)	$\partial C/\partial t$ — time decay of option value. Negative for long options positions.
Rho (ρ)	$\partial C/\partial r = KTe^{(-rT)} \cdot N(d_2)$. Interest rate sensitivity. Less critical for short-dated options.

8.3 Implied Volatility

Implied volatility (IV) is the volatility parameter σ that solves BSM(S, K, r, T, σ) = market_price. There is no closed-form solution — IV is computed numerically via Newton-Raphson iteration or Brent's method. The Platform uses the lets_be_rational library for IV computation, which achieves machine-precision results in O(1) iterations via rational function approximations to the inverse BSM function.

8.4 CVaR Mathematics

Let L be a random loss variable representing hedging P&L over a rebalancing period. Value-at-Risk at confidence level α is defined as $VaR_\alpha = \inf\{l : P(L > l) \leq 1-\alpha\}$. Conditional Value-at-Risk (CVaR), equivalently Expected Shortfall (ES), is:

CVaR Definition

$$CVaR_\alpha = E[L \mid L > VaR_\alpha]$$
$$= (1/(1-\alpha)) \cdot \int_{VaR_\alpha}^{\infty} l \cdot f_L(l)\, dl$$

For discrete sample $\{l_1, ..., l_n\}$ sorted ascending:

$$VaR_\alpha = l_{\lfloor \alpha n \rfloor}$$
$$CVaR_\alpha = (1/\lceil (1-\alpha)n \rceil) \cdot \Sigma_{i=\lfloor \alpha n \rfloor}^{n} l_i$$

CVaR is coherent in the sense of Artzner et al. (1999) — it satisfies sub-additivity, positive homogeneity, monotonicity, and translation invariance. VaR violates sub-additivity and therefore cannot be used as a risk measure for portfolio optimization. Training neural hedgers with CVaR loss functions directly optimizes for tail-risk coherence.

8.5 Deep Hedging Theory

Classical delta hedging minimizes instantaneous hedging error under BSM assumptions but performs poorly under transaction costs, discrete rebalancing, and model misspecification. Deep hedging (Buehler et al., 2019) replaces the BSM delta with a learned policy $\pi_\theta(state_t)$ that maps market state to hedge ratio. The policy is trained to minimize:

Deep Hedging Objective

$$\min_\theta CVaR_\alpha(\Sigma_t [\pi_\theta(state_t) \cdot \Delta S_t] + transaction\_costs)$$

where $\Delta S_t = S_{t+1} - S_t$ is the spot move over the rebalancing interval
and the sum telescopes to the total hedging P&L over the option lifetime

8.6 Volatility Clustering

Empirical equity returns exhibit volatility clustering — periods of high volatility persist, and periods of low volatility persist. GARCH(1,1) models this as: $\sigma^2_t = \omega + \alpha \cdot \varepsilon^2_{t-1} + \beta \cdot \sigma^2_{t-1}$. Sequential hedging models (LSTM/Transformer) are able to implicitly learn volatility regime detection from the time series of returns and option prices, allowing them to adapt hedge ratios dynamically to the current volatility regime without explicit GARCH parameterization.

8.7 Stochastic Volatility Models for Simulation

The live simulation engine supports Heston stochastic volatility dynamics for synthetic tick generation: $dS = \mu S\, dt + \sqrt{v}\, S\, dW_1$, $dv = \kappa(\theta - v)\, dt + \xi\sqrt{v}\, dW_2$, where W_1 and W_2 are correlated Brownian motions with correlation ρ. Heston ticks provide a richer training environment for neural hedgers than pure GBM, exposing models to the volatility smile and volatility-spot correlation that characterize real equity options markets.

9. Deep Dive — Machine Learning Architecture

9.1 Model Hierarchy

Model	Architecture & Role
FFNN (ffnn.py)	3–5 layer feedforward network. Stateless per-tick baseline. Maps [S, K, T, σ, Δ, Γ] → hedge ratio. Simplest benchmark.
LSTM Hedger (lstm_hedger.py)	Stacked LSTM with hidden state carried across timesteps. Processes sequences of market states. Learns temporal volatility patterns. Primary research target.
Transformer (transformer.py)	Multi-head self-attention over fixed-length market state windows. Parallel computation over sequence. Better gradient flow than LSTM for long dependencies.
Adversarial Generator	GAN-style generator network. Trained to maximize hedger CVaR loss. Produces worst-case market sequences for stress testing.

9.2 CVaR Loss Function

The custom CVaR loss in cvar_loss.py implements a differentiable approximation to the CVaR objective. The standard CVaR computation involves sorting, which is non-differentiable. The differentiable approximation uses the Rockafellar-Uryasev representation:

Differentiable CVaR Loss

$$\text{CVaR}_\alpha(L) = \min_{z \in \mathbb{R}} \{ z + (1/(1-\alpha)) \cdot E[\max(L-z, 0)] \}$$

This is differentiable with respect to L and z jointly.

In PyTorch: CVaR_loss = z + (1/(1-alpha)) * torch.mean(torch.relu(loss_batch - z))

where z is a learnable scalar parameter optimized jointly with model weights.

9.3 TorchScript Tracing

TorchScript is PyTorch's mechanism for capturing model computation graphs as portable, Python-free artifacts. trace_model.py uses torch.jit.trace(model, dummy_input) to execute the model once with sample input and record all tensor operations. The resulting ScriptModule is a static computation graph with no Python dependencies. It is serialized via model.save('deep_hedger_v1.pt') and can be loaded in any LibTorch C++ binary without a Python installation.

Tracing limitations: control flow that depends on tensor values (e.g., if x > 0) is not captured correctly. Models with data-dependent branching must use torch.jit.script instead, which compiles via Python AST analysis. The Platform uses trace for feedforward and LSTM models (no data-dependent branching in the inference path) and script for Transformer models with dynamic attention masking.

9.4 Why Python Training + C++ Inference is Optimal

Python's scientific computing ecosystem (PyTorch, NumPy, Pandas, Matplotlib, W&B) is unmatched for model development ergonomics. C++'s execution model (no GC, zero-allocation control, SIMD, cache control) is required for microsecond-range inference. The TorchScript bridge decouples these concerns

completely: researchers iterate in Python without touching C++ code, while the production engine loads new model versions from disk without code changes. This is the canonical separation of concerns for production ML systems.

9.5 Tensor Marshaling from C++ Structs

The LibTorch inference call requires populating a torch::Tensor from the C++ tick struct fields. This is performed by pre-allocating a float* buffer of the correct size (feature_dim × 1), writing struct fields into consecutive positions, and constructing the tensor via torch::from_blob(buffer, {1, feature_dim}, torch::kFloat32). from_blob creates a tensor that references the existing buffer without copying — zero-copy tensor construction.

9.6 Experiment Tracking Architecture

Every training run is initialized with wandb.init(project, config=hyperparams_dict). The hyperparams.yaml file is loaded at train.py startup and logged in full to W&B. Per-step logging includes: training CVaR loss, validation CVaR loss, gradient L2 norm, hedge ratio mean and std, and spot-hedge correlation. Model checkpoints are saved every N epochs (configurable) as W&B Artifacts with the training run ID embedded in the artifact name for full traceability.

9.7 Online vs Offline Inference

The Platform uses offline inference (pre-trained, fixed weights) in V1. The LibTorch model registry loads static .pt artifacts and executes inference without weight updates at runtime. Online learning — updating model weights in response to live market observations — is architecturally possible (LibTorch supports backward passes in C++) but is deferred to the adversarial training expansion phase (V2+) due to the stability and reproducibility guarantees required for production.

10. Deep Dive — Python / FastAPI Orchestration Plane

10.1 ARQ Async Task Queue Architecture

ARQ organizes deferred work around async Python functions registered as queue handlers. Each handler is a coroutine that receives a shared arq.ArqRedis context plus the job payload dict. The Platform registers three handler functions: run_backtest_job (historical replay), notify_model_updated (model registry refresh after new .pt artifact), and generate_report (async P&L report aggregation). Handler registration and Redis connection parameters are defined in a WorkerSettings dataclass committed to src/infra/task_queue.py.

Job enqueueing is performed inside FastAPI route handlers via await redis.enqueue_job('run_backtest_job', job_payload, _job_id=correlation_id). Using the correlationId as the ARQ job ID ensures one-to-one mapping between API requests and queue jobs, enabling idempotent re-submission and direct status lookup without a secondary index. Job status is queryable via arq.jobs.Job(job_id, redis).status() — the ARQ job record in Redis carries status (deferred/queued/in_progress/complete/not_found), result, and error fields.

Job progress is reported from within the handler coroutine via await ctx['job'].update_progress(pct, message). A dedicated ARQ progress listener task in the FastAPI lifespan subscribes to Redis keyspace notifications on job progress keys and relays updates to the relevant WebSocket subscribers. This provides the real-time progress bar in the frontend without polling. ARQ's configurable retry_jobs=True with max_tries and retry_delay parameters handles transient C++ engine failures with exponential backoff.

ARQ Queue Configuration (src/infra/task_queue.py)

```
class WorkerSettings:
  functions = [run_backtest_job, notify_model_updated, generate_report]
  redis_settings = RedisSettings(host=REDIS_HOST, port=6379)
  max_jobs = 10            # concurrent jobs per worker process
  job_timeout = 3600       # max seconds before timeout
  keep_result = 86400      # seconds to retain completed job results
  retry_jobs = True
  max_tries = 3
```

10.2 Correlation ID Propagation

Every request carries a correlationId generated at API ingress via Python's uuid.uuid4(). The correlationId propagates through all system layers: stored as the ARQ job ID in Redis; passed as a gRPC request metadata entry (x-correlation-id) on every EngineClient call; included in every structured log record as a context variable via Python's contextvars.ContextVar; persisted with the PostgreSQL job record; and embedded in every WebSocket payload. The Python contextvars mechanism — specifically a ContextVar[str] set at request entry and read by the logging configuration — ensures that all log records emitted during the handling of a given request automatically carry the correct correlationId without explicit parameter threading through the call stack.

10.3 Structured Logging — loguru

The Platform uses loguru for structured JSON logging throughout the Python backend. loguru is preferred over the standard logging module for three reasons: (1) it provides a single logger singleton (from loguru import logger) with no handler configuration boilerplate; (2) its serialize=True sink mode emits each log record as a complete JSON object to stdout without format string construction; (3) it integrates cleanly with FastAPI's request lifecycle via a middleware that binds correlationId and request metadata to the loguru context for the duration of each request.

The loguru configuration in src/config/logger.py removes the default stderr sink and adds a single stdout sink with serialize=True, level configurable via LOG_LEVEL environment variable. The JSON record schema is: { text (message), record: { time, level, name, function, line, extra: { correlationId, service, ...structuredFields } } }. All log call sites use keyword arguments for structured fields — logger.info('backtest_started', job_id=job_id, strategy=strategy) — never f-string interpolation, ensuring machine-parsable structured output.

```
# src/config/logger.py
from loguru import logger
import sys

def configure_logger(level: str = 'INFO') -> None:
    logger.remove()
    logger.add(sys.stdout, serialize=True, level=level,
               format='{time:YYYY-MM-DDTHH:mm:ss.SSS}Z | {level} | {message}')
    logger.configure(extra={'service': 'backend', 'correlationId': ''})
```

10.4 gRPC Stream Handling in Python asyncio

The infra/remote_client.py EngineClient wraps all C++ gRPC calls using the grpcio grpc.aio (asyncio-native) API. The gRPC channel is created at application startup as grpc.aio.insecure_channel(ENGINE_HOST) with keepalive parameters (GRPC_ARG_KEEPALIVE_TIME_MS=10000, GRPC_ARG_KEEPALIVE_TIMEOUT_MS=5000). The channel object is stored on AppState.live.live_client and shared across all concurrent requests without per-request channel creation overhead.

Server-streaming calls are handled via the async iterator pattern. The start_backtest coroutine calls stub.StartBacktest(request, metadata=[('x-correlation-id', correlation_id)]) and returns a grpc.aio.UnaryStreamCall object. The ARQ job handler iterates this call asynchronously:

```
async for state_update in stub.StartBacktest(request):
    payload = {
        'jobId':    request.job_id,
        'tick_ts':  state_update.tick_timestamp_ns,
        'spot':     state_update.spot_price,
        'greeks':   message_to_dict(state_update.greeks),
        'models':   [message_to_dict(m) for m in state_update.model_results],
        'cvar':     message_to_dict(state_update.cvar),
    }
    await broadcast_to_job_subscribers(request.job_id, payload)
```

The message_to_dict helper uses google.protobuf.json_format.MessageToDict with preserving_proto_field_name=True to produce a dict with snake_case field names consistent with the existing OTrader v2 frontend data contract. This zero-boilerplate conversion avoids manual field mapping and handles nested messages recursively.

gRPC error handling uses the grpc.aio.AioRpcError exception. On stream interruption, the handler logs the gRPC status code and details, marks the ARQ job as failed with the error payload, and broadcasts a JOB_ERROR event to WebSocket subscribers. Retry logic defers to ARQ's retry_jobs mechanism rather than inline reconnection, ensuring clean job state semantics.

10.5 REST Control Surface — FastAPI Endpoints

The FastAPI application exposes the following endpoint groups, organized as APIRouter instances mounted in server_fastapi.py:

Endpoint	Purpose

POST /api/gateway/connect	Connect IB gateway via C++ gRPC ConnectGateway RPC.
POST /api/gateway/disconnect	Disconnect gateway and optionally stop market data.
GET /api/gateway/status	Gateway connection status from AppState.live.live_status.
GET /api/market/status	Market data subscription status.
POST /api/market/start	Start market data via C++ gRPC StartMarketData.
POST /api/market/stop	Stop market data via C++ gRPC StopMarketData.
GET /api/strategies	List running strategies from C++ gRPC ListStrategies.
POST /api/strategies	Create strategy; body AddStrategyRequest (Pydantic).
POST /api/strategies/{name}/start	Start strategy via gRPC StartStrategy.
POST /api/strategies/{name}/stop	Stop strategy via gRPC StopStrategy.
DELETE /api/strategies/{name}/remove	Soft delete strategy via gRPC RemoveStrategy.
POST /api/run_backtest	Submit backtest job to ARQ queue. Returns 202 + jobId.
POST /api/backtest/cancel	Cancel current backtest process via AppState.backtest.
GET /api/backtest/strategies	Strategy classes available from C++ config.
GET /api/files	List .dbn / .parquet files available for backtest.
GET /api/system/status	Backend running status + live engine status.
POST /api/system/restart_live	Logical restart: Disconnect/Connect + Stop/Start MD.
GET /api/orders-trades/db	Historical orders/trades from PostgreSQL (read-only).
GET /api/database/contracts	Contract summary view from PostgreSQL.
WS /ws/logs	Real-time log subscription via FastAPI WebSocket.
WS /ws/strategies	Reserved strategy push channel (lifespan-managed connection).

11. Deep Dive — Frontend Terminal

11.1 Zustand Streaming State Management

Zustand is chosen over Redux or React Context for streaming state management due to its minimal re-render footprint. Unlike React Context, Zustand store updates do not trigger re-renders in all subscribed components — only components that subscribe to specific state slices re-render when those slices change. In a high-frequency streaming scenario (60Hz+ updates), this is critical: the P&L chart should re-render on every tick, but the configuration panel should not.

The useTradeStore.ts slice maintains: pnlHistory (array of {time, pnl} objects for all models); liveGreeks ({delta, gamma, vega, theta}); cvarHistory ({time, cvar95, cvar99}); jobStatus (idle/running/complete); connectionState (connected/disconnected). All mutations are performed via actions that append to arrays with bounded history (max 10,000 points, oldest evicted on overflow to prevent unbounded memory growth).

11.2 WebSocket Synchronization

The useWebSocket.ts hook manages a single WebSocket connection per active job. On jobId change, the hook closes the previous connection and opens a new one to ws://<backend>/stream?jobId=<id>. Incoming messages are parsed as JSON and dispatched to Zustand via updateTradeState(payload). Reconnection is implemented with exponential backoff (100ms initial, 2x multiplier, 30s cap). Connection state is reflected in the Zustand store and displayed in the terminal status bar.

11.3 High-Frequency Graph Rendering

Recharts' LineChart component is driven by the pnlHistory array from Zustand. For streaming updates at > 10Hz, direct React state-driven re-renders produce visible jitter due to React reconciliation overhead. The Platform uses a useEffect + requestAnimationFrame pattern: Zustand state changes schedule a RAF callback, which reads the latest state and updates the chart data reference. This decouples data arrival rate from render rate, ensuring renders occur at exactly 60Hz regardless of data frequency.

11.4 CVaR Histogram Visualization

CVaR visualization uses Recharts' BarChart with dynamically computed histogram bins over the cvarHistory array. Bin counts are computed in the Zustand selector using a useMemo dependency on cvarHistory. The histogram overlays a vertical line at the VaR threshold and shades the tail region in red, providing an immediate visual intuition for tail risk concentration.

11.5 Browser Memory Pressure Considerations

A 60Hz streaming session that accumulates P&L data for 1 hour produces 216,000 data points per model. With 4 models and 5 numeric fields each, this is approximately 17MB of raw float64 data — manageable, but requiring explicit management. The Platform caps all history arrays at 10,000 points (configurable). For longer sessions, the frontend requests pre-aggregated OHLC-style summaries from the backend API for historical rendering, using full-resolution data only for the last 1,000 ticks.

12. Repository & Monorepo Strategy

12.1 Repository Layout & Workspace Philosophy

The monorepo hosts three distinct language ecosystems — C++, Python, and TypeScript — each with its own package manager, toolchain, and dependency resolution strategy. There is no attempt to unify these under a single package manager; doing so creates version resolution conflicts and forces lowest-common-denominator tooling. Instead, each ecosystem owns its dependency surface completely, and the monorepo root provides only cross-cutting concerns: Docker Compose orchestration, CI/CD pipeline definitions, and the canonical engine_service.proto file from which all language-specific stubs are generated.

Dependency Manager Assignments

C++ (engines/cpp_execution_core): vcpkg in manifest mode (vcpkg.json)

Python backend (apps/backend): uv (pip-compatible, Rust-based, lock-file: uv.lock)

Python research (research/brain_layer_ml): uv (separate virtual environment)

Next.js frontend (apps/frontend): pnpm (pnpm-workspace.yaml, pnpm-lock.yaml)

12.2 Python Dependency Management — uv

Both Python components (apps/backend and research/brain_layer_ml) use uv as the package and virtual environment manager. uv is chosen over pip, poetry, and conda for three reasons: (1) resolution speed — uv resolves and installs large dependency graphs (e.g., torch + grpcio + sqlmodel) in seconds rather than minutes, materially improving Docker build cache miss latency; (2) deterministic lock files — uv.lock pins all transitive dependencies with content hashes, providing reproducibility guarantees equivalent to pnpm-lock.yaml; (3) compatibility — uv is a drop-in replacement for pip and pip-tools; existing requirements.txt files from OTrader v2 are consumed directly during migration, generating a uv.lock without requiring reformat.

Each Python component maintains a separate pyproject.toml with its own dependency groups. The backend pyproject.toml lists: fastapi, uvicorn[standard], grpcio, grpcio-tools, sqlmodel, asyncpg, alembic, arq, redis, loguru, pydantic, pyarrow, and psycopg2-binary. The research pyproject.toml lists: torch, torchvision, numpy, pandas, pyarrow, wandb, scikit-learn, and pytest. No cross-component dependencies are declared — the research environment does not import fastapi, and the backend environment does not import torch. This separation is enforced by Docker multi-stage builds that install only the relevant pyproject.toml in each image.

Virtual environment isolation is maintained locally via uv venv --python 3.11 in each component directory. A root-level Makefile provides convenience targets (make install-backend, make install-research) that activate the correct venv and run uv sync. The .python-version file at the repository root pins the CPython version (3.11.x) used by all Python components, enforced by uv's version pinning mechanism.

12.3 pnpm Workspace — Frontend Only

The pnpm workspace is scoped exclusively to the Next.js frontend (apps/frontend). The pnpm-workspace.yaml at the repository root lists only apps/frontend as a workspace package. This is a deliberate narrowing from the Node.js-inclusive V1 design: with the backend migrated to Python, there are no shared TypeScript packages, no cross-package tRPC clients, and no Node.js worker processes that would benefit from pnpm hoisting. The pnpm workspace is retained rather than migrated to npm or yarn because pnpm's strict node_modules structure (symlinked, non-flat) prevents phantom dependency issues in the Next.js build.

12.4 IDE Workspace Separation

The repository supports three distinct IDE configurations: (1) CLion for the C++ engine (engines/cpp_execution_core) — CMake integration, C++20 language services, Valgrind and TSAN runners;

(2) VSCode for the Python backend and research (apps/backend, research/brain_layer_ml) — Pylance type-checking, Python debugger, pytest runner, loguru log viewer extension; (3) VSCode for the Next.js frontend (apps/frontend) — TypeScript language services, ESLint integration, Tailwind CSS IntelliSense. Separate .code-workspace files are committed for each VSCode context to prevent Python interpreter configuration from contaminating TypeScript projects and vice versa.

12.5 Proto Stub Generation Strategy

The engine_service.proto file lives at the repository root under proto/. A Makefile target (make gen-proto) runs protoc with both grpc_cpp_plugin (for the C++ engine) and grpcio-tools grpc_tools.protoc (for the Python backend), writing generated stubs to engines/cpp_execution_core/src/proto/ and apps/backend/src/proto/ respectively. Generated stub files are committed to the repository (not gitignored) — this ensures that the CI environment does not require a protoc installation and that stub-source consistency is enforced by code review. The Makefile target is the single authoritative source for stub regeneration; ad-hoc protoc invocations are prohibited.

12.6 Docker Strategy

Docker Compose orchestrates five services: postgres (PostgreSQL 16-alpine, persistent named volume, pg_isready health check), redis (Redis 7-alpine, maxmemory 512mb, AOF enabled), cpp-engine (multi-stage: CMake+vcpkg build stage, Ubuntu 22.04 minimal runtime stage, gRPC port 50051), backend (Python 3.11-slim, uv venv with backend pyproject.toml, uvicorn entry point, port 8080, depends_on: [postgres, redis, cpp-engine]), frontend (Next.js standalone output, Node.js 20-alpine, port 3000, depends_on: [backend]).

The Python backend Docker build uses a two-stage pattern: stage 1 installs uv and runs uv sync --frozen to populate the venv from uv.lock; stage 2 copies only the venv and application source, keeping the final image free of build tools. The --frozen flag ensures Docker build fails if uv.lock is out of sync with pyproject.toml, preventing silent dependency drift between development and production images.

12.7 vcpkg & CMake Integration

The C++ engine uses vcpkg in manifest mode (vcpkg.json) for dependency management. Dependencies include: grpc, protobuf, zeromq, gtest, and libtorch. CMakeLists.txt uses find_package() for all vcpkg-managed dependencies. The vcpkg toolchain file is injected via CMAKE_TOOLCHAIN_FILE at configure time. The Docker build stage caches the vcpkg installation layer separately from the source build layer, reducing incremental build times from 15+ minutes to < 2 minutes.

13. Detailed Data Flow

13.1 Historical Replay Lifecycle

Step 1 — API Ingestion: User POSTs backtest configuration to /api/v1/backtest. Express validates the request schema (Zod), generates a UUID correlation ID, and calls bullmqProducer.add('backtest', jobData).

Step 2 — Job Dequeue: ARQ worker coroutine dequeues the job from Redis. Worker calls PostgreSQL via AsyncSession to fetch strategy metadata and validates model paths exist on the filesystem.

Step 3 — Parquet Loading: Worker uses the Apache Arrow Node.js library to open the Parquet file and stream row batches. Each batch is serialized to a Protobuf StreamRequest and sent to the C++ engine via gRPC.

Step 4 — C++ Replay Loop: The C++ engine receives the StreamRequest, populates the ZMQ-equivalent tick stream from the Protobuf payload, and processes ticks through the full execution loop as described in Section 7.

Step 5 — State Streaming: Each tick produces an EngineStateUpdate streamed back to Node.js via gRPC server-side streaming. The Node.js worker receives updates on the ClientReadableStream 'data' event.

Step 6 — WebSocket Broadcast: Each update is transformed to JSON and broadcast to all WebSocket clients subscribed to the jobId.

Step 7 — Result Persistence: On stream completion, the worker computes summary statistics and writes the backtest result record to PostgreSQL.

13.2 ML Training Lifecycle

Step 1 — Dataset Preparation: data_loader.py opens Parquet files via memory-mapped I/O (pyarrow.memory_map). A custom PyTorch Dataset wraps the memory-mapped RecordBatch with lazy slice loading. DataLoader provides batching, shuffling, and multi-process prefetching.

Step 2 — Training Loop: train.py initializes the model, optimizer (Adam or AdamW), and CVaR loss function. For each epoch, the DataLoader yields batches. Forward pass computes hedge ratios. CVaR loss is computed from the batch P&L distribution. Backward pass computes gradients. Optimizer steps.

Step 3 — Experiment Logging: wandb.log() records per-step metrics. Model checkpoints are saved every N epochs.

Step 4 — TorchScript Export: trace_model.py loads the best checkpoint, constructs a dummy input tensor matching the model's expected input shape, calls torch.jit.trace(model, dummy_input), and saves the resulting ScriptModule as a .pt file.

Step 5 — Registry Update: The .pt file is written to the shared artifacts directory. The model registry entry in PostgreSQL is updated with the new artifact path and training run metadata.

13.3 WebSocket Broadcast Lifecycle

The broadcast pipeline maintains sub-5ms end-to-end latency from gRPC receipt to WebSocket delivery. The critical path is: gRPC ClientReadableStream 'data' event (async I/O, handled by libuv event loop) → deserialization from Protobuf binary to JS object (< 100µs for typical payload) → JSON.stringify() of broadcast payload (< 50µs) → WebSocket ws.send() for each subscriber (< 500µs per client for buffered socket). Total for single subscriber: ~1ms. For 100 concurrent subscribers: ~2–3ms due to single-threaded Node.js serialization but parallel OS socket writes.

14. Detailed Sequence Narratives

14.1 Historical Backtest Launch Sequence

Sequence: Historical Backtest Launch

1. [UI → API] POST /api/v1/backtest { strategy, models, dateRange, instrument }
2. [API] Validate request schema; generate correlationId; create DB job record (PENDING)
3. [API → ARQ] Enqueue job via redis.enqueue_job(_job_id=correlationId)
4. [API → UI] 202 Accepted { jobId, correlationId }
5. [UI → WS] WebSocket connect ws://backend/ws/stream?jobId=<jobId>
6. [ARQ → Worker] Job dequeued; ARQ worker coroutine claims job (status: in_progress in Redis)
7. [Worker → DB] Fetch strategy metadata; validate model registry entries
8. [Worker → C++] gRPC StartBacktest(StreamRequest) — open server-side stream
9. [C++ Engine] Load Parquet batch; populate tick buffer; begin execution loop
10. [C++ → Worker] gRPC Stream: EngineStateUpdate { tick, pnl, delta, cvar, models[] }
11. [Worker → WS] Broadcast JSON payload to all jobId subscribers
12. [WS → UI] ws.onmessage → Zustand updateTradeState → Recharts repaint
13. [Steps 9–12 repeat for every tick in the dataset]
14. [C++ → Worker] gRPC stream END signal
15. [Worker → DB] Persist summary statistics; update job status to COMPLETE
16. [Worker → WS] Broadcast JOB_COMPLETE event

14.2 Live Tick Processing Sequence

Sequence: Live Tick Processing

1. [ZMQ Feed] ZeroMQ PUB socket emits tick: { ts_ns, bid, ask, last, iv }
2. [ZMQ Thread] zmq_recv() returns; deserialize tick struct from wire bytes
3. [ZMQ Thread] SPSC push: atomic write_idx increment; tick slot filled
4. [Exec Loop] Spin-poll: detects write_idx advance; SPSC pop tick
5. [Exec Loop] Black-Scholes: compute d1, d2, call/put prices, full Greeks
6. [Exec Loop] LibTorch: populate input tensor from tick fields; forward() on each model
7. [Exec Loop] CVaR: append P&L to sliding window; compute tail statistics
8. [Exec Loop] EngineStateUpdate struct populated on stack; SPSC push to outbound
9. [gRPC Thread] SPSC pop state; Protobuf serialize; ServerWriter::Write() call
10. [gRPC Network] Protobuf frame transmitted to Node.js gRPC client channel

Total expected latency (steps 1–10): < 200 microseconds for LSTM inference

14.3 TorchScript Deployment Sequence

Sequence: TorchScript Deployment

1. [Research] train.py completes training; saves best checkpoint

2. [Research] trace_model.py: model.eval(); dummy_input constructed
3. [Research] torch.jit.trace(model, dummy_input) → ScriptModule
4. [Research] script_module.save('artifacts/deep_hedger_v1.pt')
5. [API Call] POST /api/v1/models/load { modelId: 'deep_hedger_v1', path: '...' }
6. [Node.js] Validate artifact path exists; create DB model registry entry
7. [Node.js → C++] gRPC LoadModel command via MPSC command buffer
8. [C++ Registry] torch::jit::load(path) → validate input/output shapes
9. [C++ Registry] Register module in registry map under modelId key
10. [C++ → Node.js] gRPC MODEL_LOADED acknowledgment
11. [Node.js → WS] Broadcast { event: 'modelLoaded', modelId } to UI

14.4 CVaR Risk Update Pipeline

Sequence: CVaR Update Pipeline

1. [Exec Loop] Tick processed; hedging P&L delta computed for each model
2. [Exec Loop] Append P&L delta to circular window buffer (size=1000 ticks)
3. [Exec Loop] If window full: sort window ascending (insertion sort, O(N) amortized)
4. [Exec Loop] VaR_95 = window[950]; CVaR_95 = mean(window[950:1000])
5. [Exec Loop] VaR_99 = window[990]; CVaR_99 = mean(window[990:1000])
6. [Exec Loop] Populate CVaR fields in EngineStateUpdate
7. [gRPC Thread] Emit CVaR fields alongside Greeks and P&L in same frame
8. [Node.js] Relay to WebSocket broadcast layer
9. [UI] Zustand cvarHistory updated; CVaR histogram re-renders

15. API & IPC Contract Design

15.1 Protobuf Schema Philosophy

Protobuf is chosen over JSON for the gRPC IPC layer for three reasons: (1) binary encoding produces messages 3–10x smaller than equivalent JSON, reducing serialization/deserialization CPU cost and network transfer time; (2) the .proto schema is a machine-enforceable contract — fields are typed, required/optional semantics are explicit, and schema evolution has well-defined backward compatibility rules; (3) generated code from protoc eliminates manual serialization boilerplate and guarantees cross-language type safety.

15.2 Example Protobuf Schema

```
syntax = "proto3";
package engine;

service EngineService {
  rpc StartBacktest(StreamRequest) returns (stream EngineStateUpdate);
  rpc SendCommand(stream CommandRequest) returns (CommandAck);
}

message StreamRequest {
  string job_id = 1;
  string correlation_id = 2;
  repeated string model_ids = 3;
  StrategyConfig strategy = 4;
}

message EngineStateUpdate {
  string job_id = 1;
  int64 tick_timestamp_ns = 2;
  double spot_price = 3;
  double implied_vol = 4;
  GreeksPayload greeks = 5;
  repeated ModelResult model_results = 6;
  CVaRPayload cvar = 7;
}

message ModelResult {
  string model_id = 1;
  double hedge_ratio = 2;
  double pnl = 3;
  double cumulative_pnl = 4;
  int64 inference_latency_ns = 5;
}

message CVaRPayload {
  double var_95 = 1;
  double cvar_95 = 2;
  double var_99 = 3;
  double cvar_99 = 4;
}
```

15.3 Versioning Strategy

Proto files are versioned with the package namespace (engine.v1, engine.v2). Field numbers are never reused after deprecation — deprecated fields are tagged with [deprecated=true] and removed only in major version bumps. Node.js and C++ protoc-generated bindings are regenerated and committed to the repository on every .proto change, ensuring generated code is always in sync with the schema.

16. Database & Persistence Design

16.1 ORM Strategy — SQLModel with AsyncPG

The persistence layer uses SQLModel as the ORM, providing a dual-purpose class hierarchy that serves simultaneously as SQLAlchemy table definitions and Pydantic API response models. This eliminates the schema duplication overhead of maintaining separate SQLAlchemy models and Pydantic schemas — a single SQLModel class annotated with table=True is both the database table and the serializable response type. SQLModel is built on SQLAlchemy 2.0, inheriting its async session support, and on Pydantic v2, inheriting its validation and serialization performance improvements.

All database I/O uses SQLAlchemy's async session API (sqlalchemy.ext.asyncio.AsyncSession) with the asyncpg driver. This ensures database calls are non-blocking within the FastAPI/uvicorn event loop — a SELECT query does not block the OS thread, allowing the event loop to service concurrent WebSocket broadcasts and gRPC stream iterations during database round-trips. The async engine is created once at application startup and stored on AppState; sessions are obtained per-request via a get_session FastAPI dependency.

```
# src/db/engine.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

engine = create_async_engine(
    f'postgresql+asyncpg://{DB_USER}:{DB_PASS}@{DB_HOST}/{DB_NAME}',
    pool_size=10, max_overflow=20, pool_pre_ping=True,
)
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession,
                                 expire_on_commit=False)
```

16.2 SQLModel Table Definitions

Each database entity is defined as a SQLModel class with table=True. Field types use Python native types annotated with Optional and Field() for defaults, constraints, and JSON column handling. JSONB columns (parameters, summary, input_shape) use SQLAlchemy's JSON type with PostgreSQL's native JSONB storage class for indexable structured data.

Table / SQLModel Class	Key Columns & Design Notes
Strategy	id: UUID PK (default_factory=uuid4), name: str, instrument: str, parameters: dict (JSONB), created_at: datetime (server_default)
BacktestJob	id: UUID PK, strategy_id: UUID FK, status: str (PENDING/RUNNING/COMPLETE/FAILED/CANCELLED), correlation_id: str (unique index), started_at: datetime, completed_at: datetime | None, summary: dict | None (JSONB)
ModelRegistry	id: str PK (e.g. 'deep_hedger_v1'), torchscript_path: str, training_run_id: str, input_shape: list (JSONB), validation_cvar: float, registered_at: datetime
TickArchive	id: int PK (serial), job_id: UUID FK, tick_timestamp_ns: int (bigint), spot: float, iv: float, metadata: dict | None (JSONB)
RiskSnapshot	id: int PK (serial), job_id: UUID FK, model_id: str, tick_idx: int, delta: float, gamma: float, cvar_95: float, cvar_99: float, pnl: float

16.3 Alembic Migration Management

Schema migrations are managed by Alembic, configured to use the async SQLAlchemy engine via alembic.ini and env.py. Migration scripts are generated by alembic revision --autogenerate, which compares the live database schema against the current SQLModel table definitions and emits the diff as a Python migration script. Migration scripts are committed to the repository under alembic/versions/ and applied at container startup via alembic upgrade head in the Docker CMD. This ensures the database schema is always in sync with the application code in every deployment, eliminating the manual schema management overhead of raw SQL DDL scripts.

16.4 Redis Usage Model

Redis serves three roles in the Python backend: (1) ARQ job state — sorted sets and hashes for job lifecycle (deferred/queued/in_progress/complete), progress values, and results; (2) WebSocket fan-out coordination — Redis PubSub channels keyed by jobId allow multiple uvicorn worker processes to broadcast to their respective local WebSocket client sets without a centralized registry; (3) model registry cache — string KV pairs mapping model IDs to SQLModel-serialized metadata dicts with TTL=3600s, avoiding repeated PostgreSQL round-trips for the model listing endpoint. Redis is configured with maxmemory-policy allkeys-lru to prevent unbounded memory growth. AOF persistence is enabled for ARQ job state durability across Redis restarts.

16.5 Read-Only DB Queries — psycopg2 for Legacy Views

The OTrader v2 codebase includes a set of read-only PostgreSQL queries in src/infra/db.py (fetch_orders_trades_raw, fetch_contracts_summary) implemented using psycopg2 with raw SQL. These queries target schema views that are populated by the C++ engine's trade execution path — not by the Python application's SQLModel tables. These psycopg2-based queries are retained as-is during the Platform migration: they are synchronous, called from a FastAPI endpoint with run_in_executor to avoid event loop blocking, and do not require SQLModel or SQLAlchemy model definitions because they return raw dicts shaped directly for the frontend.

16.6 Parquet Storage

Historical tick data is stored as columnar Apache Parquet files partitioned by date and instrument, with the directory structure data/{symbol}/{YYYY}/{MM}/{DD}.parquet. The schema includes: timestamp_ns (int64), bid (float64), ask (float64), last (float64), volume (int64), implied_vol (float64), strike (float64), expiry_date (date32). The backtest_runner.run_backtest_cpp function resolves parquet paths by listing the data/{symbol}/ directory, filtering by date range, and passing the resolved paths as CLI arguments to the entry_backtest C++ executable. Parquet files are read directly by the C++ executable using the Arrow C++ library — the Python layer only resolves paths, never reads file contents.

17. Infrastructure & DevOps

17.1 Docker Compose Architecture

The docker-compose.yml defines the following service topology: postgres (image: postgres:16-alpine, persistent named volume, health check via pg_isready), redis (image: redis:7-alpine, maxmemory 512mb, AOF enabled), cpp-engine (multi-stage build: stage 1 vcpkg+cmake build, stage 2 Ubuntu 22.04 minimal runtime, exposes gRPC port 50051), backend (Python 3.11-slim, two-stage uv build: stage 1 installs uv and runs uv sync --frozen, stage 2 copies venv + source, uvicorn entry on port 8080, depends_on: [postgres, redis, cpp-engine]), arq-worker (same Python image as backend, entry point: python -m arq src.infra.task_queue.WorkerSettings, depends_on: [redis, cpp-engine]), frontend (Next.js standalone output, Node.js 20-alpine, port 3000, depends_on: [backend]).

The arq-worker is deployed as a separate Compose service from the FastAPI backend, sharing the same Docker image but with a different CMD. This separation allows the ARQ worker process to be scaled independently of the uvicorn HTTP/WebSocket process, and ensures that a worker crash does not affect the API's ability to accept new job submissions. Both services are connected to the internal Docker bridge network and share access to the same Redis instance for job queue coordination.

Service health checks: cpp-engine uses grpc_health_probe (gRPC Health Check RPC); backend uses curl localhost:8080/api/system/status; arq-worker uses a custom check script that queries arq job counts via Redis CLI. Compose restart policy is on-failure with max_attempts=3.

17.2 CI/CD Strategy

The CI/CD pipeline (GitHub Actions) executes on every push: (1) C++ build and test: cmake configure with vcpkg toolchain, cmake --build, ctest with GoogleTest suite, Google Benchmark results stored as artifacts; (2) Python backend lint and test: uv run ruff check src/, uv run mypy src/, uv run pytest tests/ with coverage; (3) Python research test: uv run pytest research/brain_layer_ml/tests; (4) Next.js lint, typecheck, build: pnpm run lint, pnpm run typecheck, pnpm run build; (5) Docker build: docker build for each service with layer cache via buildx. Integration tests (grpc.aio mock, WebSocket throughput via locust or httpx-ws) run on merge to main. Release builds push Docker images tagged with git SHA to the container registry.

17.3 Observability Stack

V1 observability uses loguru's structured JSON stdout output, captured by Docker's logging driver and forwarded to a log aggregator (Loki or OpenSearch). V2 will add prometheus_fastapi_instrumentator to expose /metrics from the FastAPI backend (request latency histograms, active WebSocket connections, ARQ queue depth) and Grafana dashboards for full stack observability. The C++ engine will expose Prometheus metrics via a lightweight HTTP endpoint using the prometheus-cpp library, providing execution loop latency histograms and ring buffer utilization gauges alongside the Python service metrics.

18. Testing & Validation Strategy

18.1 C++ Testing — GoogleTest

test_math.cpp validates Black-Scholes pricing and Greeks against known analytical solutions: ATM call price at T=1, σ=0.2 is verified against reference implementation to 8 decimal places. Put-call parity C - P = S·e^{-qT} - K·e^{-rT} is verified for 1000 random (S, K, T, σ, r) samples. Delta bounds (0 ≤ Δ_call ≤ 1) and Gamma non-negativity are asserted across all samples.

test_buffers.cpp stress-tests SPSC and MPSC ring buffers under concurrent load: producer thread pushes 10M items at maximum speed; consumer thread pops concurrently; test verifies zero items dropped, zero items duplicated, and correct ordering. Valgrind memcheck run detects any memory errors. Thread sanitizer (TSAN) run detects data races.

18.2 Python Testing — Pytest

test_loss.py verifies: CVaR gradient flows correctly through the Rockafellar-Uryasev formulation using torch.autograd.gradcheck; CVaR of a deterministic loss distribution equals the known analytical value; CVaR at α=0 equals mean loss; CVaR at α→1 equals maximum loss. Model forward passes are verified for correct output shape and no NaN/Inf outputs across 100 random inputs.

18.3 Python Backend Testing — Pytest + httpx + pytest-asyncio

The FastAPI application is tested using pytest with the pytest-asyncio plugin for coroutine test functions and httpx.AsyncClient for async HTTP testing. The test suite covers: (1) API integration tests — AsyncClient(app=app, base_url='http://test') submits POST /api/run_backtest, verifies 202 with valid UUID correlationId, verifies PostgreSQL BacktestJob record created with PENDING status; (2) WebSocket tests — httpx-ws WebSocketSession connects to /ws/logs, verifies log messages broadcast from a mock log_queue injection; (3) gRPC mock tests — a pytest fixture starts a mock grpc.aio server implementing StartBacktest, verifies the EngineClient correctly iterates 1000 EngineStateUpdate messages and calls broadcast_to_job_subscribers for each; (4) ARQ queue tests — verify that enqueue_job with a valid payload creates an ARQ job record in Redis with correct status; (5) SQLModel CRUD tests — verify Strategy, BacktestJob, and ModelRegistry insert/select/update round-trips using an in-memory SQLite engine (via aiosqlite driver, compatible with SQLModel's async session API).

18.4 Replay Determinism Tests

A dedicated determinism test suite runs the full historical replay pipeline twice with identical inputs and verifies SHA-256 checksums of all emitted EngineStateUpdates are identical between runs. This test is run in CI and gates all merges. Any non-determinism detected triggers a blocking investigation — the test cannot be bypassed.

18.5 Latency Benchmarking

C++ microbenchmarks (Google Benchmark) measure: SPSC push/pop throughput (expected: > 100M ops/sec); Black-Scholes evaluation (expected: < 1µs per call); LibTorch FFNN inference (expected: < 50µs); LSTM inference (expected: < 200µs). Results are stored in the CI artifact store and compared against previous run baselines — regressions > 10% block the merge.

19. Dataset Strategy

19.1 Dataset Overview

Dataset	Purpose	Integration Point
Databento	Institutional-grade OPRA options tick data. Bid/ask/last at millisecond resolution.	Primary research training dataset. Parquet ingestion via Arrow.
IBKR TWS	Live brokerage feed for real-money simulation. Options chains with Greeks.	ZeroMQ adapter for live simulation mode.
ThetaData	Historical options end-of-day and intraday data. Cost-effective bulk ingestion.	Supplementary training data. CSV → Parquet conversion pipeline.
Polygon.io	Equities and options aggregates. REST + WebSocket APIs.	Baseline equity data for underlying dynamics modeling.
Kaggle HFT	Synthetic HFT order book data. Useful for microstructure noise modeling.	Research exploration. Not used in primary training.
Optiver Datasets	Realized volatility prediction competition data. Option-implied vol surfaces.	Feature engineering research. Vol surface interpolation training.
BANKNIFTY Options	Indian index options. High liquidity, exotic microstructure characteristics.	Cross-market generalization research. Emerging market hedging models.

19.2 Preprocessing Pipeline

All datasets pass through a common preprocessing pipeline before training: (1) timestamp normalization to UTC nanosecond integers; (2) bid-ask spread outlier removal (spread > 5% of mid rejected); (3) implied volatility computation via Newton-Raphson for datasets providing only option prices; (4) session boundary labeling (pre-market, regular, after-hours); (5) conversion to Apache Parquet with standardized schema; (6) checksum validation to ensure data integrity. Preprocessing scripts are idempotent and versioned.

20. Security & Reliability

20.1 API Hardening

Express API is hardened with: Helmet.js security headers (CSP, HSTS, X-Frame-Options); rate limiting via express-rate-limit (100 requests/15min per IP); request body size limit (1MB); schema validation via Zod on all input routes; no stack traces in production error responses. gRPC server validates Protobuf message field constraints via custom interceptor before dispatching to handlers.

20.2 WebSocket Authentication

WebSocket upgrade requests are authenticated via JWT token passed as a query parameter or Authorization header. Tokens are validated against the same secret as REST API tokens. Unauthenticated upgrade requests are rejected with HTTP 401 before the WebSocket handshake completes. Per-connection rate limiting prevents a single client from flooding the subscription system.

20.3 Replay Integrity

Historical tick files are accompanied by SHA-256 checksums stored in the PostgreSQL tick_archives table. Before a replay job begins, the worker verifies the checksum of each Parquet file against the stored value. Checksum mismatch blocks the job with a DATA_INTEGRITY_ERROR status and triggers an alert.

20.4 Process Crash Isolation

The C++ engine, FastAPI uvicorn server, and ARQ worker are separate OS processes. A crash in the execution engine does not crash the backend API. Docker restart policies ensure crashed containers are restarted within 5 seconds. ARQ jobs in progress at the time of a worker crash are automatically re-queued after the stall timeout (configurable, default 30 seconds) — ARQ's at-least-once delivery guarantee is backed by Redis TTL on the in-progress job key. The frontend degrades gracefully on WebSocket disconnection with a reconnection UI indicator.

21. Scalability Strategy

21.1 Worker Replication

ARQ worker processes are stateless and horizontally scalable. Additional arq worker instances can be deployed on separate VMs or containers, all consuming from the same Redis-backed queue. ARQ's job locking mechanism (Redis SET NX with TTL) ensures each job is processed by exactly one worker. Worker count can be scaled dynamically based on Redis queue depth metrics exposed via prometheus_fastapi_instrumentator.

21.2 WebSocket Scaling

Multiple WebSocket broadcast server instances coordinate via Redis PubSub. The gRPC stream handler publishes each EngineStateUpdate to a Redis channel keyed by jobId. All WS server instances subscribe to the relevant channel and broadcast to their local client set. This provides horizontal scaling of the WebSocket tier without centralized bottlenecks.

21.3 Future Kubernetes Migration

The Docker Compose architecture is designed to be Kubernetes-compatible. Each Compose service maps directly to a Kubernetes Deployment. Stateful services (PostgreSQL, Redis) migrate to StatefulSets or managed cloud equivalents (RDS, ElastiCache). The FastAPI backend Deployment and the ARQ worker Deployment share a Docker image but use different CMD overrides — uvicorn for the API pod, python -m arq for the worker pod. Horizontal Pod Autoscaler (HPA) can scale ARQ worker replicas based on Redis queue depth exposed via a custom Prometheus metric. The C++ engine, due to its statefulness and CPU affinity requirements, is deployed as a DaemonSet with nodeSelector targeting high-performance compute nodes with CPU isolation configured via Linux cgroups.

21.4 Future GPU Acceleration

LibTorch supports CUDA inference natively. Migrating from CPU to GPU inference requires: (1) changing tensor device from torch::kCPU to torch::kCUDA; (2) moving pre-allocated input tensors to GPU memory at startup; (3) ensuring output tensors are copied back to CPU for downstream processing. GPU inference latency for the LSTM model is expected to decrease from ~200µs (CPU) to ~20µs (CUDA), enabling faster rebalancing frequencies.

22. Technical Risks

Risk	Category	Mitigation Strategy
gRPC backpressure causing buffer overflow	Performance	Outbound SPSC buffer with configurable drop-on-overflow; backpressure metrics exported; grpc.aio flow control via channel options
LibTorch inference NaN/Inf on adversarial inputs	ML Stability	Output validation gate in execution loop; fallback to Black-Scholes on invalid inference; comprehensive test suite covering boundary inputs
Replay non-determinism from floating-point platform differences	Correctness	Pin to specific CPU microarchitecture in Docker; use deterministic FP mode (no fast-math); SHA-256 regression tests
Memory fragmentation in long-running C++ process	Memory	Jemalloc allocator; all hot-path allocations pre-allocated; periodic heap profiling in CI
Race condition in MPSC command buffer under high command rate	Concurrency	Formal verification of MPSC implementation; TSAN in CI; bounded command rate at API layer
Model drift: production distribution shift from training data	ML Ops	Continuous validation set CVaR monitoring; alerting on drift threshold breach; periodic retraining pipeline
FastAPI WebSocket broadcast blocking event loop under high fan-out	Scalability	asyncio.gather for concurrent sends; WebSocketDisconnect evicts slow clients; Redis PubSub for multi-instance fan-out
ARQ worker starvation under high backtest concurrency	Queue	max_jobs=1 per engine instance prevents oversubscription; ARQ job_timeout enforces bounded execution; monitoring of Redis queue depth
asyncpg connection pool exhaustion under burst writes	Persistence	pool_size and max_overflow tuned; async session lifecycle scoped per request; batched INSERT for risk snapshots
Parquet schema mismatch across dataset versions	Data Quality	Schema validation at ingestion with Pandera; versioned schema registry in PostgreSQL; preprocessing idempotency
TorchScript tracing incomplete for data-dependent control flow	ML Portability	Use torch.jit.script for branching models; comprehensive tracing validation test suite; CI validation of .pt artifact load

23. Roadmap & Milestones

Phase 1 — Workspace Scaffolding & Core Plumbing

Objectives: Establish the complete monorepo structure, toolchain configurations, Docker Compose topology, and inter-service communication scaffolding.

Deliverables: pnpm workspace configured; Docker Compose with all five services; gRPC proto file defined; C++ CMakeLists.txt building successfully with vcpkg dependencies; Node.js Express server with /health endpoint; Next.js shell with WebSocket hook stub.

Acceptance Criteria: docker-compose up brings all services to healthy state. gRPC connectivity test passes (C++ echo service responds to Node.js client). pnpm install from root resolves all dependencies without conflicts.

Engineering Validation Gate: CI pipeline green on all services; gRPC connectivity integration test passing.

Phase 2 — C++ Mathematical Baseline

Objectives: Implement production-quality Black-Scholes engine, SPSC/MPSC ring buffers, ZMQ tick ingestion, and gRPC state streaming.

Deliverables: black_scholes.hpp with all five Greeks; spsc_ring_buffer.hpp and mpsc_ring_buffer.hpp; zmq_consumer.hpp ingesting synthetic ticks; grpc_server.cpp streaming EngineStateUpdates; backtest_engine.cpp processing static tick file.

Acceptance Criteria: GoogleTest suite passes all mathematical truth-set validations. SPSC throughput > 100M ops/sec under TSAN. gRPC stream emits > 10,000 updates/sec to Node.js client. P&L at option expiry equals intrinsic value within 1e-6 tolerance.

Engineering Validation Gate: All C++ tests passing; latency benchmarks within budget; no TSAN or Valgrind errors.

Phase 3 — Python FastAPI Orchestration

Objectives: Implement the FastAPI application structure, ARQ async task queue, grpc.aio engine client, FastAPI-native WebSocket broadcast layer, SQLModel PostgreSQL persistence, and loguru structured logging — migrating and extending the OTrader v2 backend codebase to serve the Platform's gRPC streaming and deep hedging use cases.

Deliverables: server_fastapi.py with FastAPI lifespan managing gRPC channel and background tasks; src/api/backtest.py run_backtest and cancel endpoints; src/infra/remote_client.py EngineClient with grpc.aio channel and async iterator stream handling; src/infra/task_queue.py ARQ WorkerSettings with run_backtest_job handler; src/infra/websockets.py log_queue_processor and job_stream_processor broadcast tasks; SQLModel table classes (Strategy, BacktestJob, ModelRegistry, RiskSnapshot) with Alembic migrations; loguru logger configured for structured JSON stdout output with correlationId ContextVar binding; Pydantic request schemas for all backtest and strategy endpoints.

Acceptance Criteria: End-to-end backtest job completes (POST /api/run_backtest → ARQ → grpc.aio stream → WS broadcast). 1000 WebSocket messages delivered to connected client with < 1% loss. correlationId present in all log records across ARQ job, gRPC call, and WebSocket broadcast. Alembic upgrade head succeeds on clean PostgreSQL instance. ARQ job retry correctly re-enqueues on simulated gRPC disconnection. loguru serialize=True output passes JSON schema validation.

Engineering Validation Gate: pytest suite covers: ARQ job enqueueing and status query; FastAPI endpoint 202/422/404 responses; grpc.aio mock streaming (10,000 messages with zero loss); SQLModel CRUD operations on all table classes; Alembic migration round-trip (upgrade + downgrade).

Phase 4 — Frontend Terminal

Objectives: Build the real-time trading terminal with Zustand state management, Recharts P&L and CVaR visualization, and configuration panel.

Deliverables: useWebSocket.ts hook with reconnection; useTradeStore.ts Zustand slice; pnl-chart.tsx Recharts canvas; risk-panel.tsx CVaR histogram; Shadcn configuration form for backtest submission.

Acceptance Criteria: P&L chart updates at 60Hz during live backtest. CVaR histogram correctly reflects tail distribution. Configuration panel submits valid backtest request and displays job progress. No browser memory leak over 60-minute session.

Phase 5 — Integration & Stress Testing

Objectives: Full end-to-end integration testing, latency profiling, WebSocket throughput stress testing, and replay determinism validation.

Deliverables: Determinism test suite (SHA-256 replay comparison); gRPC mock for Node.js integration tests; Autocannon WebSocket stress test scripts; p99 latency profile report.

Acceptance Criteria: All determinism tests pass. WebSocket sustains 10,000 msg/sec to 50 concurrent clients without message loss. p99 tick-to-broadcast latency < 5ms. Zero critical bugs in backtest reconciliation.

Phase 6 — ML Research & LibTorch Embedding

Objectives: Train FFNN, LSTM, and Transformer hedging models; implement CVaR loss; integrate LibTorch model registry into C++ execution core; enable multi-model benchmarking.

Deliverables: cvar_loss.py with gradient tests; lstm_hedger.py and transformer.py training to convergence; trace_model.py exporting valid .pt artifacts; LibTorch registry in C++ loading and executing all model types; W&B experiment dashboard.

Acceptance Criteria: All three models train to validation CVaR < Black-Scholes benchmark on held-out test set. TorchScript artifacts load successfully in C++ and execute without NaN/Inf. Multi-model benchmark comparison visible in frontend terminal.

24. Future Expansion

24.1 Reinforcement Learning-Based Hedging

Deep hedging can be reformulated as a Markov Decision Process and solved via reinforcement learning. The hedge ratio at each timestep is an action; the negative CVaR of the terminal P&L is the reward. Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC) can be applied to learn hedging policies that adapt dynamically to regime changes. The Platform's research workspace is architecturally prepared for RL: the simulation environment already produces the state/action/reward tuples required by standard RL frameworks.

24.2 FPGA Exploration

For ultra-high-frequency applications (nanosecond latency), FPGA acceleration of Black-Scholes computation and order routing represents the frontier. The Platform's lock-free architecture and strictly typed Protobuf IPC are compatible with FPGA PCIe DMA transfer patterns. An FPGA co-processor could replace the software Black-Scholes implementation with a pipelined hardware implementation achieving sub-100ns latency.

24.3 Multi-Asset Support

The current architecture targets single-underlying options. Extension to multi-asset portfolios requires: (1) a portfolio-level CVaR computation that aggregates correlated P&L distributions; (2) cross-Greeks (correlation Greeks) in the state update payload; (3) multi-instrument Parquet schemas; (4) a portfolio optimizer layer between the model output and the hedge execution decision. The gRPC schema is extensible to support multi-asset payloads without breaking changes.

24.4 Smart Order Routing & Exchange Connectivity

Live trading requires integration with exchange gateways via FIX protocol or proprietary APIs. The ZMQ tick consumer can be replaced with a FIX session manager that receives market data and sends orders. The execution loop's hedge ratio output can be connected to an order management system that translates delta hedges into equity/futures orders with slippage models and market impact constraints.

25. Final Technical Conclusion

25.1 Institutional Credibility

The Platform's architectural decisions — lock-free ring buffers, CPU affinity pinning, TorchScript portability, gRPC typed IPC, deterministic replay, CVaR-optimized training — are not cosmetic choices made for resume value. Each decision solves a specific, well-characterized problem in the quantitative trading infrastructure space that naive implementations invariably fail on at scale. The architecture is consistent with internal systems at Two Sigma, Jane Street, and Citadel in its separation of concerns, performance orientation, and infrastructure-first philosophy.

25.2 Why Infrastructure-First Sequencing is Correct

The most common failure mode in quant research projects is the model-first, infrastructure-never trap: research prototypes in Jupyter notebooks achieve excellent backtest performance, fail at productionization, and are eventually abandoned. By building the execution core, IPC layer, orchestration plane, and observability stack before optimizing model architectures, the Platform ensures that model improvements are immediately deployable and that production performance exactly matches research validation. Infrastructure investment is never wasted — it compounds across every subsequent research cycle.

25.3 Why Low-Latency + ML Convergence Matters

The convergence of low-latency systems engineering and machine learning is not merely a technical curiosity — it represents a genuine competitive moat. A deep hedging model that runs at 200µs per inference decision can rebalance 5,000 times per second, adapting to microstructure dynamics that are invisible to systems operating at the 10ms+ latency characteristic of Python-based inference pipelines. The performance envelope of the Platform's C++/LibTorch hybrid architecture creates a class of hedging strategies that are architecturally unavailable to competitors using conventional Python/REST-based infrastructure.

25.4 Portfolio-Grade Assessment

The Platform demonstrates mastery across six engineering domains simultaneously: (1) low-latency systems (C++20, lock-free, zero-allocation); (2) distributed systems (gRPC, ARQ, Redis, PostgreSQL); (3) machine learning engineering (PyTorch, TorchScript, CVaR, adversarial training); (4) frontend engineering (Zustand, Recharts, high-frequency rendering); (5) quantitative finance (Black-Scholes, Greeks, CVaR, deep hedging theory); (6) DevOps (Docker, CI/CD, uv/vcpkg/pnpm reproducibility, observability). The migration from Node.js to Python for the orchestration plane adds a seventh dimension of architectural sophistication: the deliberate maximization of code reuse from a production-validated codebase (OTrader v2), demonstrating not just the ability to build systems from scratch but to extend and integrate existing institutional infrastructure with minimal disruption. The Platform is a portfolio-defining artifact that credibly demonstrates institutional-grade engineering capability.

— END OF DOCUMENT —

Classification: INTERNAL — RESTRICTED | Version 2.0.0 | June 2026